

Advanced programming Assignment

ROLL NO: CSB24065

NAME: Gourisankar Bhuyan

Session: Spring

Year:2026

Question:

Develop an activity log analyzer in python:

You are given a list of activity records:

```
{ "user": str, //roll numbers of students  
  "action": str, //Online activities of students such as - apps, websites visited etc.  
  "duration": float //screen time for each activity  
}
```

You must: 1. Store data efficiently using python built-in containers.

2. Implement the following: `def total_time_per_user(logs: list[dict]) -> dict[str, float]` `def most_active_users(logs: list[dict], k: int) -> list[str]` `def unique_actions(logs: list[dict]) -> set[str]`

3.

- a. Use dict, set, and list
 - b. Use comprehensions where appropriate
 - c. Use `sorted()` with key
 - d. Avoid explicit loops where possible
 - e. Use typing annotations
 - f. Use `defaultdict` optionally
 - g. Use `reduce()` to compute total activity time.
4. Perform complexity analysis:

a. Time complexity for computing top K users

b. Space complexity of storing intermediate results

Answer:

```
from typing import List, Dict, Set
from collections import defaultdict
from functools import reduce

def total_time_per_user(logs: List[Dict[str, object]]) -> Dict[str, float]:
    def reducer(acc: Dict[str, float], log: Dict[str, object]) -> Dict[str, float]:
        acc[log["user"]] = acc.get(log["user"], 0.0) + float(log["duration"])
        return acc

    return reduce(reducer, logs, {})
```

```
def most_active_users(logs: List[Dict[str, object]], k: int) -> List[str]:
    totals = total_time_per_user(logs)

    sorted_users = sorted(
        totals.items(),
        key=lambda item: item[1],
        reverse=True
    )

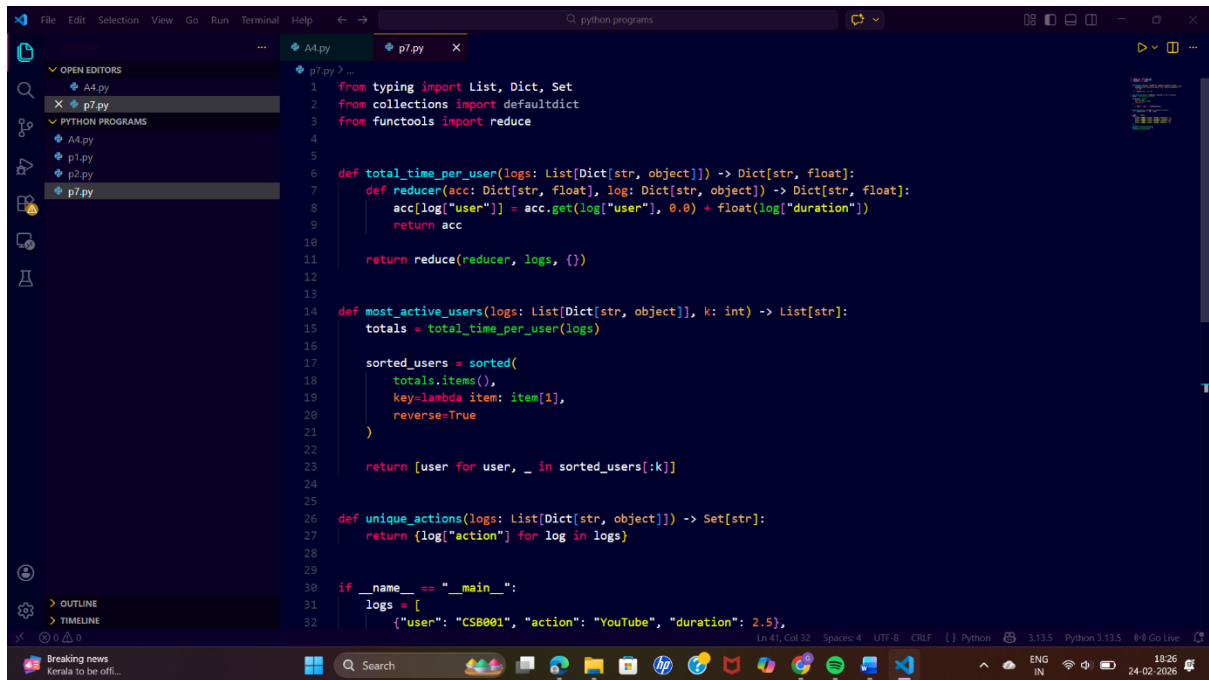
    return [user for user, _ in sorted_users[:k]]
```

```
def unique_actions(logs: List[Dict[str, object]]) -> Set[str]:
    return {log["action"] for log in logs}
```

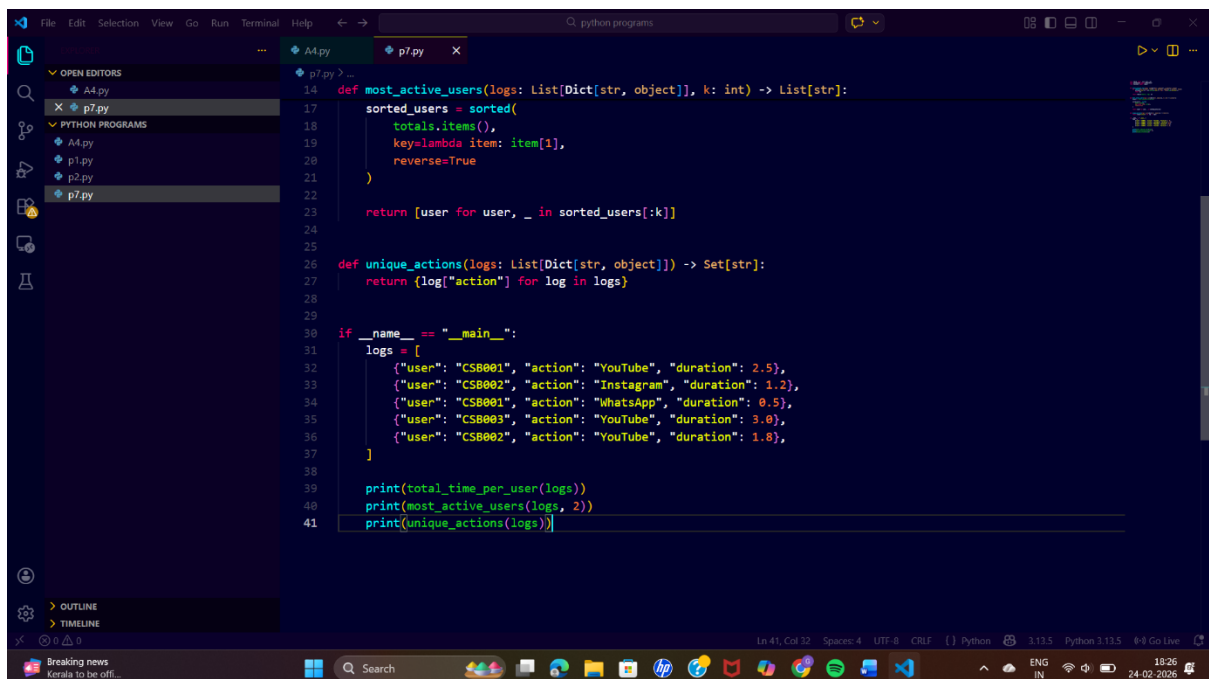
```
if __name__ == "__main__":
    logs = [
        {"user": "CSB001", "action": "YouTube", "duration": 2.5},
        {"user": "CSB002", "action": "Instagram", "duration": 1.2},
        {"user": "CSB001", "action": "WhatsApp", "duration": 0.5},
        {"user": "CSB003", "action": "YouTube", "duration": 3.0},
        {"user": "CSB002", "action": "YouTube", "duration": 1.8},
    ]

    print(total_time_per_user(logs))
    print(most_active_users(logs, 2))
    print(unique_actions(logs))
```

source code and it's outputs:



```
1 from typing import List, Dict, Set
2 from collections import defaultdict
3 from functools import reduce
4
5
6 def total_time_per_user(logs: List[Dict[str, object]]) -> Dict[str, float]:
7     def reducer(acc: Dict[str, float], log: Dict[str, object]) -> Dict[str, float]:
8         acc[log["user"]] = acc.get(log["user"], 0.0) + float(log["duration"])
9         return acc
10
11     return reduce(reducer, logs, {})
12
13
14 def most_active_users(logs: List[Dict[str, object]], k: int) -> List[str]:
15     totals = total_time_per_user(logs)
16
17     sorted_users = sorted(
18         totals.items(),
19         key=lambda item: item[1],
20         reverse=True
21     )
22
23     return [user for user, _ in sorted_users[:k]]
24
25
26 def unique_actions(logs: List[Dict[str, object]]) -> Set[str]:
27     return {log["action"] for log in logs}
28
29
30 if __name__ == "__main__":
31     logs = [
32         {"user": "CSB001", "action": "YouTube", "duration": 2.5},
```



```
14 def most_active_users(logs: List[Dict[str, object]], k: int) -> List[str]:
15     sorted_users = sorted(
16         totals.items(),
17         key=lambda item: item[1],
18         reverse=True
19     )
20
21     return [user for user, _ in sorted_users[:k]]
22
23
24
25
26 def unique_actions(logs: List[Dict[str, object]]) -> Set[str]:
27     return {log["action"] for log in logs}
28
29
30 if __name__ == "__main__":
31     logs = [
32         {"user": "CSB001", "action": "YouTube", "duration": 2.5},
33         {"user": "CSB002", "action": "Instagram", "duration": 1.2},
34         {"user": "CSB001", "action": "WhatsApp", "duration": 0.5},
35         {"user": "CSB003", "action": "YouTube", "duration": 3.0},
36         {"user": "CSB002", "action": "YouTube", "duration": 1.8},
37     ]
38
39     print(total_time_per_user(logs))
40     print(most_active_users(logs, 2))
41     print(unique_actions(logs))
```

The screenshot shows a VS Code editor with a Python file named `p7.py` open. The code defines two functions: `total_time_per_user` and `most_active_users`. The `total_time_per_user` function uses a reducer to calculate the total time for each user. The `most_active_users` function uses the `total_time_per_user` function to calculate the total time for each user and then returns the top `k` users.

```
1 from typing import List, Dict, Set
2 from collections import defaultdict
3 from functools import reduce
4
5
6 def total_time_per_user(logs: List[Dict[str, object]]) -> Dict[str, float]:
7     def reducer(acc: Dict[str, float], log: Dict[str, object]) -> Dict[str, float]:
8         acc[log["user"]] = acc.get(log["user"], 0.0) + float(log["duration"])
9         return acc
10
11     return reduce(reducer, logs, {})
12
13
14 def most_active_users(logs: List[Dict[str, object]], k: int) -> List[str]:
15     totals = total_time_per_user(logs)
16
```

The terminal output shows the execution of the script. It runs the `total_time_per_user` function on a list of logs and then the `most_active_users` function on the same logs, returning the top 3 users.

```
PS C:\Users\gouri\OneDrive\Desktop\python programs> python -u "c:\Users\gouri\OneDrive\Desktop\python programs\p7.py"
[{'CS8801': 3.0, 'CS8802': 3.0, 'CS8803': 3.0}]
[('Instagram', 'YouTube', 'WhatsApp')]
PS C:\Users\gouri\OneDrive\Desktop\python programs> python -u "c:\Users\gouri\OneDrive\Desktop\python programs\p7.py"
[{'CS8801': 3.0, 'CS8802': 3.0, 'CS8803': 3.0}]
[('Instagram', 'YouTube', 'WhatsApp')]
PS C:\Users\gouri\OneDrive\Desktop\python programs>
```

Complexity analysis:

Let:

- N = number of log records
- U = number of unique users

1 Time Complexity for Computing Top K Users

Steps involved:

1. Computing total time per user
 - Iterates through all logs once
 - Time complexity:
$$O(N)$$
2. Sorting users by total time
 - If there are U unique users, sorting requires:
$$O(U \log U)$$
3. Extracting top K
 - Slicing first K elements:
$$O(K)$$

Final Time Complexity:

$$O(N + U \log U)$$

In worst case, if every log belongs to a different user:

$$O(N \log N)$$

2 Space Complexity

- Storing totals dictionary $\rightarrow O(U)$
- Storing sorted list $\rightarrow O(U)$
- Storing unique actions $\rightarrow O(A)$
(where A = number of unique actions)

Overall Space Complexity:

$$O(U + A)$$

In worst case:

$$O(N)$$